

LLM Behavioral Audit Report

Sample Report (Abridged)

Subject	Customer Service Agent
Date	2026-04-30
Models	GPT-4o-mini, Claude Haiku 4.5, Gemini 2.5 Flash
Toolchain	llm-failure-atlas v0.1.5 + agent-failure-debugger v0.4.0

AUDIT VERDICT: FAIL

This verdict reflects behavioral failures observed across all tested models under controlled scenarios. It is not a claim about every possible deployment context.

This audit distinguishes between behavioral failures, upstream data issues, and configuration gaps — rather than treating all failures as model errors.

Primary failure (blocking): Critical pattern detected in user-facing flows.

Secondary failure: 1 check(s) classified as FAILED.

What this audit can and cannot detect

This audit is a behavioral-side diagnostic. We observe what the agent did, not what it knew. Within that scope, our detection has four distinct evidence classes, and our confidence varies by class.

Evidence class	What we observe	Confidence
Direct observation	Tool data flow, output produced, termination mode, structural failure patterns	High — deterministic from trace
Proxy	Word-overlap alignment, retrieval utilisation ratio, expansion ratio	Medium — heuristic, with named limits
Requires upstream context	Whether retrieved chunks match the right category, whether facts are correct	Outside our scope — needs partner data
Requires runtime intervention	Whether recursion limits exist, whether guardrails fire, whether degraded paths recover	Architectural — diagnosis, not detection

1. Audit Criteria

An agent must meet **all** conditions to pass:

Criterion	Threshold	Result
No FAILED scenarios	0 failed checks (adjusted)	FAIL (1 failed)
Adjusted healthy rate	$\geq 50\%$	PASS (50%)
No critical patterns	No critical-priority findings	FAIL (detected)

2. Executive Summary

This audit evaluated 6 scenarios across 3 models, producing 18 diagnostic checks.

This report is intended to guide engineering, product, and platform teams in identifying where failures originate and how to address them.

Metric	Raw	Adjusted	Note
HEALTHY	3	9	6 reclassified from false degradations
DEGRADED	14	8	Genuine quality concerns
FAILED	1	1	Agent could not complete the task
Healthy rate	17%	50%	Adjusted rate excludes alignment heuristic false positives

Scenario	GPT-4o-mini	Claude Haiku 4.5	Gemini 2.5 Flash
Order Status Lookup	HEALTHY (raw: degraded)	HEALTHY	HEALTHY (raw: degraded)
Refund Request — System Down	DEGRADED	DEGRADED	FAILED
Wrong Product Recommendation	HEALTHY	HEALTHY	HEALTHY (raw: degraded)
Policy Search Loop	DEGRADED — <code>incorrect_output</code>	DEGRADED	DEGRADED — <code>incorrect_output</code>
Cancellation Pivot to Upsell	DEGRADED — <code>incorrect_output</code>	DEGRADED — <code>incorrect_output</code>	DEGRADED — <code>incorrect_output</code>
Simple Price Check	HEALTHY (raw: degraded)	HEALTHY (raw: degraded)	HEALTHY (raw: degraded)

Example Finding (Excerpt)

P0 — Fabricated Response After Tool Failure (S2)

Trigger: Backend service returned an error; agent responded using ungrounded general knowledge without disclosing the outage.

(The full report includes detection methodology, agent response sample, business impact assessment, evidence-class tagging, and remediation actions for all P0/P1/P2 findings.)

This is an abridged sample. The full report includes:

- Priority-ranked findings (P0 / P1 / P2) with business impact assessment, agent response samples, detection methodology, remediation actions with multiple paths where applicable, and team ownership assignments
- Evidence Map showing which evidence class each finding draws from

- *When to Trust Our Signals* — per-signal trust framework with corroboration logic and resolution paths to upstream systems
- *Cross-model observations and detection coverage notes*

Methodology

Agent traces were analyzed using deterministic causal analysis (no ML). 17 failure patterns and 34 signals were evaluated per trace, organised into four evidence classes that distinguish direct observations from proxies, upstream-required context, and runtime-intervention concerns. All processing runs locally on the client's infrastructure. No traces or customer data are transmitted externally.

Generated with llm-failure-atlas v0.1.5 + agent-failure-debugger v0.4.0 — github.com/kiyoshisasano/llm-failure-atlas | github.com/kiyoshisasano/agent-failure-debugger | github.com/kiyoshisasano/pytest-agent-health